

1

1.1 Sizeof und Arrays

a)

Das Array wird mit vom Typ `int` Initialisiert. Eine Variable vom Typ `int` benötigt 4 Byte Speicherplatz. Somit braucht das Array insgesamt 4 Byte * 8 = 32 Byte Speicherplatz.

0.5/0.5

b)

`sizeof(a)` gibt die Gesamtgröße des Arrays in Byte zurück. (Siehe a) => es ergibt sich eine Gesamtgröße von 32 Byte. Um die Anzahl der Elemente im Array zu berechnen kann man die Gesamtgröße des Arrays durch die Größe eines einzelnen Elements teilen: `sizeof(a) / sizeof(a[0])` . Dies ergibt 32 Byte/4 Byte = 8 Elemente.

```
int a [8];
int i;

for (i = 0; i < sizeof(a) / sizeof(a[0]); i++) {
    a[i] = 1;
}
```

1/1

c)

Der das Ergebnis wird `8` sein, da die Operation die Speichergröße des Datentyps des Pointers zurück gibt.

0.5/0.5

d)

Ein möglichkeit wäre die Übergabe der Arraygröße als zusätzlichen Parameter an die Funktion, da in C "Array Decay" ein

GBR - Übung01 - Yunis Drescher (827130), Joshua Neunert (822456), Konrad Gose (828440)

Array in einen Pointer umwandelt, wenn es an eine Funktion übergeben wird.

richtig, aber wie würde die konkrete Berechnung aussehen

1/1

1.2

a)

Header	Meaning	Erklärung
%MEM	Memory usage	Der prozentuale Anteil des gesamten physischen Arbeitsspeichers, den der Prozess nutzt
TT	Terminal	Das Terminal, mit dem der Prozess verknüpft ist
STAT	Process state	Der aktuelle Zustand des Prozesses
STARTED	Start time/date	Wann der Prozess gestartet wurde
TIME	CPU time	Die gesamte CPU-Zeit, die der Prozess bisher verwendet hat
CMD	Command	Der Befehl (und seine Argumente), mit dem der Prozess gestartet wurde

1/1

1.3

a)

Header	Meaning	Erklärung
USER	User	Der Benutzername des Eigentümers des Prozesses
PID	Process ID	Die eindeutige Identifikationsnummer des Prozesses
PPID	Parent Process ID	Die Prozess-ID des übergeordneten Prozesses
%CPU	CPU usage	Der prozentuale Anteil der CPU-Zeit, den der Prozess seit seinem Start verwendet hat

3/3

GBR - Übung01 - Yunis Drescher (827130), Joshua Neunert (822456), Konrad Gose (828440)

Aufgabe 1.5: 6/7

dl_proc_list.h 1/1

```
typedef struct dl_proc_node {
    int pid;
    int ppid;
    struct dl_proc_node* prev;
    struct dl_proc_node* next;
} dl_proc_node;

typedef struct dl_proc_list {
    dl_proc_node* head;
} dl_proc_list;

dl_proc_list* dl_proc_list_create(void);

int dl_proc_list_insert(dl_proc_list *list, int pid);

int dl_proc_list_get(dl_proc_list *list, int position, int* pid, int* ppid);

int dl_proc_list_remove(dl_proc_list *list, int pid);

void dl_proc_list_free(dl_proc_list *list);
```

Makefile vorhanden. 0.5/0,5

workdir.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

char *mywdfunc(unsigned int bufsize) {
    if (bufsize < 100) {
        return NULL;
    }

    char *buf = (char *)calloc(bufsize,sizeof(char));
    if (buf) {
        char *cwd = getcwd(buf,bufsize-1);
        if (!cwd) {
            free(buf);
            return NULL;
        }
    }

    return buf;
}

int main() {
    char *ret = mywdfunc(512);

    printf("%s", ret);
    free(ret);
    return EXIT_SUCCESS;
}
```

Rückgabewert von mywdfunc() ebenfalls auf NULL prüfen, um Fehlerfall zu erkennen. -1

Header	Meaning	Erklärung
%MEM	Memory usage	Der prozentuale Anteil des gesamten physischen Arbeitsspeichers, den der Prozess nutzt
TT	Terminal	Das Terminal, mit dem der Prozess verknüpft ist
STAT	Process state	Der aktuelle Zustand des Prozesses
STARTED	Start time/date	Wann der Prozess gestartet wurde
TIME	CPU time	Die gesamte CPU-Zeit, die der Prozess bisher verwendet hat
CMD	Command	Der Befehl (und seine Argumente), mit dem der Prozess gestartet wurde

1/1

b)

Der Prozess `3787` hat den Statuscode `S+`, welcher sich wie folgt aufschlüsseln lässt:

S: interruptible sleep (waiting for an event to complete)
+: is in the foreground process group

- Der Prozess schläft und wartet auf Ereignisse (evt. Benutzereingabe)
- Er ist interaktiv und gehört zur ordergrundgruppe des Terminals

1/1

c)

```
--+ 0956 root login -- mschroetter
\--+ 1009 mschroe -zsh
\--+ 1929 mschroe tmux new-session
\--+ 3828 mschroe -zsh
\--- 3967 mschroe ps -eo user,pid,ppid
```

1/1

GBR - Übung01 - Yunis Drescher (827130), Joshua Neunert (822456), Konrad Gose (828440)

dl_proc_list.c 3/3,5

```
#include "dl_proc_list.h"
#include <stdlib.h>

// Creates a new process list with the initial process (PID = 1, PPID = 0)
dl_proc_list* dl_proc_list_create() {
    // Create list
    dl_proc_list *list;
    list = (dl_proc_list *) malloc(sizeof(dl_proc_list));

    if (!list) {
        return NULL;
    }

    // Create first process node
    dl_proc_node *first_node;
    first_node = (dl_proc_node *) malloc(sizeof(dl_proc_node));

    if (!first_node) {
        free(list);
        return NULL;
    }

    // Initialize first process node (PID = 1, PPID = 0)
    first_node->pid = 1;
    first_node->ppid = 0;

    first_node->prev = NULL;
    first_node->next = NULL;

    // Set head of list to first node
    list->head = first_node;

    return list;
}

// Inserts a new process with given PID into process list
// Returns 1 on success, -1 on memory allocation failure,
// -2 on invalid args, -3 if PID already exists
int dl_proc_list_insert(dl_proc_list *list, int pid){
    // Invalid list
    if (!list) {
        return -2;
    }

    // Invalid PID
    if (pid <= 1) {
        return -2;
    }

    // PID already exists
    if ((pre_new && pre_new->pid == pid) || (pre_new && pre_new->next && pre_new->next->pid == pid)) {
        return -3;
    }

    // pre_new is either the end of the list or the last node with PID < new PID
    // Invalid position
    if (!pre_new) {
        return -3;
    }

    // Create new process node
    dl_proc_node *new_node = (dl_proc_node *) malloc(sizeof(dl_proc_node));
    if (!new_node) {
        return -1;
    }

    new_node->pid = pid;
    new_node->ppid = pre_new->ppid;
    new_node->next = pre_new->next;
    new_node->prev = pre_new;

    if (pre_new->next) {
        pre_new->next->prev = new_node;
    }
    pre_new->next = new_node;

    return 1;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define SIZE 512

int main() {
    int fd = open("./test.txt", O_RDWR);
    if (fd == -1){
        perror("could not open file");
        return EXIT_FAILURE;
    }

    char * buffer = (char *) calloc(SIZE, sizeof(char));

    if (!buffer) {
        perror("could not allocate memory");
        close(fd);
        return EXIT_FAILURE;
    }

    ssize_t r = read(fd, buffer, SIZE-1);

    if (r == -1) {
        perror("could not read file");
        free(buffer);
        close(fd);
        return EXIT_FAILURE;
    }

    printf("%s", buffer);
    free(buffer);
    close(fd);
    return EXIT_SUCCESS;
}
```

Es können weitere Elemente in der Liste existieren, deren PID größer ist als von pre_new->next.

Liefert nicht notwendigerweise die bisher höchste Prozess-ID -0.5

larger inserted after smaller

larger inserted after smaller

Aufgabe 1.4: 2/3

print_file.c

```
#include <stdio.h>
#include <fcntl.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

#define SIZE 512

int main() {
    int fd = open("./test.txt", O_RDWR);
    if (fd == -1){
        perror("could not open file");
        return EXIT_FAILURE;
    }

    char * buffer = (char *) calloc(SIZE, sizeof(char));

    if (!buffer) {
        perror("could not allocate memory");
        close(fd);
        return EXIT_FAILURE;
    }

    ssize_t r = read(fd, buffer, SIZE-1);

    if (r == -1) {
        perror("could not read file");
        free(buffer);
        close(fd);
        return EXIT_FAILURE;
    }

    printf("%s", buffer);
    free(buffer);
    close(fd);
    return EXIT_SUCCESS;
}
```

```

    return 0;
}

int dl_proc_list_get(dl_proc_list *list, int position, int*
pid, int* ppid){
    // Invalid list
    if (!list) {
        return -1;
    }

    dl_proc_node *current = list->head;
    int index = 0;

    while (current && index < position) {
        current = current->next;
        index++;
    }

    // Position out of bounds
    if (!current) {
        return -2;
    }

    *pid = current->pid;
    *ppid = current->ppid;

    return 0;
}

int dl_proc_list_remove(dl_proc_list *list, int pid){
    // Invalid list
    if (!list) {
        return -2;
    }

    // Invalid PID, cannot remove PID 1
    if (pid <= 1) {
        return -2;
    }

    dl_proc_node *current = list->head;

    while (current && current->pid != pid) {
        current = current->next;
    }

    // PID not found
    if (!current) {
        return -3;
    }
}

```

```

    }

    int parent_pid = current->ppid;

    // Remove the node from the list
    if (current->prev) {
        current->prev->next = current->next;
    } else {
        // Removing head node
        list->head = current->next;
    }

    if (current->next) {
        current->next->prev = current->prev;
    }

    free(current);
    dl_proc_node *node = list->head;
    while (node) {
        if (node->ppid == pid) {
            node->ppid = parent_pid;
        }
        node = node->next;
    }

    return 0;
}

void dl_proc_list_free(dl_proc_list *list){
    if (!list) {
        return;
    }

    dl_proc_node *current = list->head;
    dl_proc_node *next_node;

    while (current) {
        next_node = current->next;
        free(current);
        current = next_node;
    }

    free(list);
}

```

proctable.c 1,5/2

```

#include <stdio.h>
#include <stdlib.h>
#include "dl_proc_list.h"

int main(int argc, char *argv[]) {
    dl_proc_list *plist = dl_proc_list_create();
    if (!plist) {
        fprintf(stderr, "Fehler: Konnte Prozessstabelle nicht
erstellen.\n");
        return EXIT_FAILURE;
    }

    for (int i = 1; i < argc; i++) {
        char *endptr;
        long val = strtol(argv[i], &endptr, 10);

        if (*endptr != '\0') {
            fprintf(stderr, "Warnung: Ungultiges Argument
'%s' ignoriert.\n", argv[i]);
            continue;
        }

        if (val == 0) {
            fprintf(stderr, "Warnung: PID = 0 wird ignoriert.
\n");
            continue;
        }

        if (val > 0) {
            if (dl_proc_list_insert(plist, (int)val) < 0) {
                fprintf(stderr, "Fehler beim Einfuegen: PID
%d existiert bereits.\n", val);
            }
        } else {
            int pid_to_remove = (int)(-val);
            if (pid_to_remove == 1) {
                fprintf(stderr, "Fehler: Prozess mit ID = 1
kann nicht geloescht werden.\n");
            } else if (dl_proc_list_remove(plist,
pid_to_remove) < 0) {
                fprintf(stderr, "Fehler beim Loeschen: Prozess
mit ID = %d existiert nicht.\n", pid_to_remove);
            }
        }
    }

    int pos = 0, pid, ppid;
    while (dl_proc_list_get(plist, pos, &pid, &ppid) == 0) {
        printf("PID = %d, Parent PID = %d\n", pid, ppid);
        pos++;
    }
}

```

strtoll() liefert einen Wert vom Typ long zurück. Vor dem Casting nach int darauf achten, dass der Wert nicht unter INT_MIN bzw. über INT_MAX liegt. -0,5

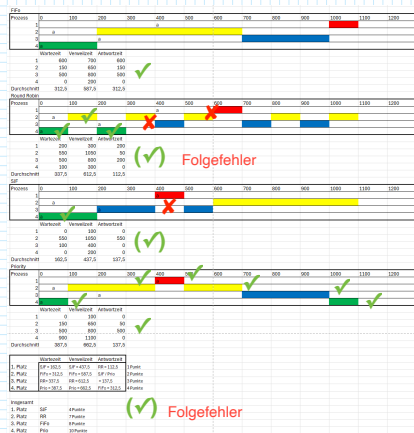
Übungsblatt 2

Dienstag, 13. November 2025 22:28

Yunis Drescher, 827330 Joshua Neunert, 822456 Konrad Gose, 828440

10/11 Aufgabe 2.2: Scheduling

- 5/5 a) In der Übung wurde die FIFO-Scheduling-Strategie in den Sched-Sim Simulator implementiert. Erweitern Sie `scheduler.c` um die Implementierung der Round-Robin, Shortest-Job-First- und Fixed-Priority-Scheduling-Strategie. Erweitern Sie falls nötig die Implementierung der Run-Queue in `queue.c` um die benötigten Funktionen. (5 Punkte)
- 2/3 b) Erstellen Sie für die Algorithmen Round-Robin, Shortest-Job-First und Fixed-Priority je ein Gantt-Diagramm, wobei die Kosten für Scheduler und Dispatcher vernachlässigt werden sollen. Überprüfen Sie die Laufzeit und Ablaufzeiten der Prozesse aus der `main.c`.
Hinweis: Verwenden Sie für Ihre Zeichnung ein Quantum von 100ms. (3 Punkte)
- 3/3 c) Bewerten Sie die Effizienz der drei Scheduling Strategien in Bezug auf die mittlere Wartezeit, die mittlere Verweilzeit und die mittlere Antwortzeit. Benutzen Sie sich dabei auf die in Aufgabe b) erstellten Ablaufdiagramme. (3 Punkte)



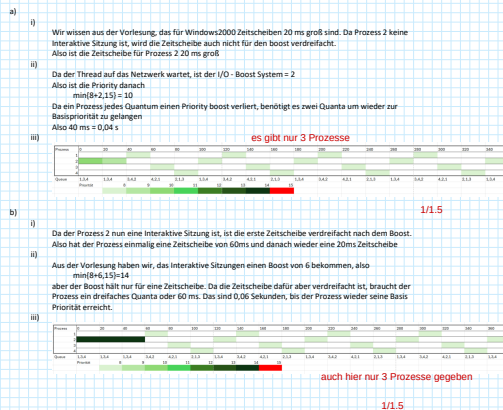
Aufgabe 2.3: Windows Scheduler

Gegeben sei ein **Eingeprosessor-System** mit Windows 2000. Weiterhin existieren 3 Benutzerprozesse mit der regulären **BasePriority**, von denen zwei rechenintensiv sind. Bearbeiten Sie die drei nachfolgenden Szenarien mit den dazugehörigen Teilaufgaben.

- a) Im ersten Szenario ist (Prozess 2) auf das Netzwerk wartend blockiert. Prozess 2 erhält zum Zeitpunkt $t=0$ die CPU, wenn das Netzwerk bereit ist.
- Ermitteln Sie die Größe der Zeitscheibe von Prozess 2.
 - Ermitteln Sie die Anzahl an Sekunden, bis Prozess 2 seine **BasePriority** wieder erreicht hat.
 - Zeichnen Sie ein modifiziertes Gantt-Diagramm, aus dem die aktuellen Prioritäten der Prozesse ablesbar sind. Das Diagramm muss für jeden Prozess mindestens 4 Zeitscheiben umfassen.
- b) Im zweiten Szenario ist Prozess 2 ein GUI-Thread. Bearbeiten Sie die Teilaufgaben a)(i)-(iii) erneut für diesen Fall.

Hinweis: Benutzen Sie die Boost- und Zeitscheibengrößen aus der Vorlesung!

(1,5 + 1,5 Punkte)



zu b)

Round Robin: P3 wird bei Ankunft bei 200 ms vor P2 eingeordnet und startet ab 300 ms. -0,5 Analog wird bei der Ankunft von P1 vorgegangen, wodurch P1 bereits ab 500 ms startet.

Die richtige Reihenfolge lautet also: P4, P2, P4, P3, P2, P1, ...

SJF: P3 wird nicht durch die Ankunft von P1 bei 400 ms unterbrochen. -0,5 SJF arbeitet nicht-verdrängend.

a) 5/5

queue.c

```

#include <queue.h>
#include <stdlib.h>
#include <stddef.h>
#include <process.h>
#include <schedule.h>
#include <internal/misc.h>

int queue_enqueue_rr(process_queue_t *queue, process_t
*process);
int queue_enqueue_sjf(process_queue_t *queue, process_t
*process);
int queue_enqueue_prio(process_queue_t *queue, process_t
*process);

void queue_set_functions(void)
{
    // setting queue functions
    global_state.queue_functions.queue_init = queue_init;
    global_state.queue_functions.queue_enqueue =
queue_enqueue;
    global_state.queue_functions.queue_destroy =
queue_destroy;
    global_state.queue_functions.queue_length = queue_length;
    global_state.queue_functions.queue_get_process_at =
queue_get_process_at;

    // If you want that new processes are added in a different
way for one scheduler
    if (schedule == schedule_sjf)
    {
        global_state.queue_functions.queue_enqueue =
queue_enqueue_sjf;
    }
    else if (schedule == schedule_prio)
    {
        global_state.queue_functions.queue_enqueue =
queue_enqueue_prio;
    }
    else if (schedule == schedule_rr)
    {
        global_state.queue_functions.queue_enqueue =
queue_enqueue_rr;
    }

    return;
}

```

```

int queue_init(process_queue_t **queue)
{
    *queue = (process_queue_t

```

```

*)malloc(sizeof(process_queue_t));
if (*queue == NULL)
{
    return -1;
}
(*queue)->head = NULL;
(*queue)->tail = NULL;
(*queue)->length = 0;
return 0;
}

// Default
int queue_enqueue(process_queue_t *queue, process_t *process)
{
    process_node_t *new_node = (process_node_t *)malloc(sizeof(process_node_t));
    if (new_node == NULL)
    {
        return -1;
    }
    new_node->process = process;
    new_node->prev = queue->tail;
    new_node->next = NULL;

    if (queue->tail)
    {
        queue->tail->next = new_node;
    }
    else
    {
        queue->head = new_node;
    }
    queue->tail = new_node;
    queue->length++;
    return 0;
}

// Round Robin
int queue_enqueue_rr(process_queue_t *queue, process_t *process)
{
    // args validation
    if (!queue || !process)
    {
        return -1;
    }

    process_node_t *new_node = (process_node_t *)malloc(sizeof(process_node_t));
    if (new_node == NULL)

```

```

{
    return -1;
}

new_node->process = process;
new_node->prev = queue->tail;
new_node->next = NULL;

if (queue->tail)
{
    // Add to the end
    queue->tail->next = new_node;
}
else
{
    // Default set as head
    queue->head = new_node;
}

queue->tail = new_node;
queue->length++;
return 0;
}

// Shortest-Job-First
int queue_enqueue_sjf(process_queue_t *queue, process_t *process)
{
    // args validation
    if (!queue || !process)
    {
        return -1;
    }

    process_node_t *new_node = (process_node_t *)malloc(sizeof(process_node_t));
    if (new_node == NULL)
    {
        return -1;
    }

    new_node->process = process;
    new_node->prev = NULL;
    new_node->next = NULL;

    if (queue->head == NULL)
    {
        // Insert as Head
        queue->head = new_node;
        queue->tail = new_node;
    }
}

```

```

}
else
{
    process_node_t *current = queue->head;
    while (current != NULL && current->process->burst_time <= process->burst_time)
    {
        current = current->next;
    }
    if (current == queue->head)
    {
        new_node->next = queue->head;
        queue->head->prev = new_node;
        queue->head = new_node;
    }
    else if (current == NULL)
    {
        new_node->prev = queue->tail;
        queue->tail->next = new_node;
        queue->tail = new_node;
    }
    else
    {
        new_node->next = current;
        new_node->prev = current->prev;
        current->prev->next = new_node;
        current->prev = new_node;
    }
}
queue->length++;
return 0;
}

// Fixed-Priority
int queue_enqueue_prio(process_queue_t *queue, process_t *process)
{
    process_node_t *new_node = (process_node_t *)malloc(sizeof(process_node_t));
    if (new_node == NULL)
    {
        return -1;
    }

    new_node->process = process;
    new_node->prev = NULL;
    new_node->next = NULL;

    if (queue->head == NULL)
    {

```

```

queue->head = new_node;
queue->tail = new_node;
}
else
{
    process_node_t *current = queue->head;
    while (current != NULL && current->process->priority <= process->priority)
    {
        current = current->next;
    }
    if (current == queue->head)
    {
        new_node->next = queue->head;
        queue->head->prev = new_node;
        queue->head = new_node;
    }
    else if (current == NULL)
    {
        new_node->prev = queue->tail;
        queue->tail->next = new_node;
        queue->tail = new_node;
    }
    else
    {
        new_node->next = current;
        new_node->prev = current->prev;
        current->prev->next = new_node;
        current->prev = new_node;
    }
}
queue->length++;
return 0;
}

int queue_dequeue(process_queue_t *queue, process_t **process)
{
    if (queue->head == NULL)
    {
        return -1;
    }
    process_node_t *node_to_remove = queue->head;
    *process = node_to_remove->process;
    queue->head = node_to_remove->next;

    if (queue->head)
    {
        queue->head->prev = NULL;
    }
    else

```

```

{
    queue->tail = NULL;
}
free(node_to_remove);
queue->length--;
return 0;
}

int queue_length(process_queue_t *queue)
{
    return queue->length;
}

int queue_get_process_at(process_queue_t *queue, process_t **process, int index)
{
    if (index < 0 || index >= (int)queue->length)
    {
        return -1;
    }
    process_node_t *current = queue->head;
    for (int i = 0; i < index; i++)
    {
        current = current->next;
    }
    *process = current->process;
    return 0;
}

int queue_destroy(process_queue_t *queue)
{
    process_node_t *current = queue->head;
    while (current != NULL)
    {
        process_node_t *next = current->next;
        free(current);
        current = next;
    }
    free(queue);
    return 0;
}

```

schedule.c

```

#include <schedule.h>
#include <process.h>
#include <queue.h>
#include <internal/misc.h>

int schedule_fifo(process_t *current_process, process_t **next_process)
{
    if (current_process && current_process->state == READY) {
        *next_process = current_process;
        return 0;
    }

    if (queue_length(current_core->run_queue) < 1) {
        return -1;
    }

    return queue_dequeue(current_core->run_queue, next_process);
}

// Round Robin
int schedule_rr(process_t *current_process, process_t **next_process)
{
    int ret;
    process_t *next = NULL;

    // first element in the run queue
    ret = queue_dequeue(current_core->run_queue, &next);

    if (ret == 0) {
        // append the current process back to the run queue if it's still READY
        if (current_process && current_process->state == READY) {
            int enq_ret = queue_enqueue(current_core->run_queue, current_process);
            if (enq_ret != 0) {
                return enq_ret;
            }
        }

        // assign the next process to run
        *next_process = next;
        return 0;
    }
    else {
        // empty queue

        // let the current process continue if it's still READY
        if (current_process && current_process->state == READY) {
            *next_process = current_process;

```

```

        return 0;
    }
}

// queue is empty
return -1;

// Shortest Job First
int schedule_sjf(process_t *current_process, process_t **next_process)
{
    int len = 0;
    // If a process is running continue
    if (current_process && current_process->state == READY) {
        *next_process = current_process;
        return 0;
    }
    // If no process is running find next
    if ((len = queue_length(current_core->run_queue)) < 1) {
        return -1;
    }

    process_t *shortest_job = NULL;
    process_t *temp_process = NULL;
    int ret = 0;

    // find shortest job in the run queue
    for (int i = 0; i < len; i++) {
        ret = queue_dequeue(current_core->run_queue, &temp_process);
        if (ret != 0) {
            return ret;
        }

        // Check if process is new shortest job
        if (shortest_job == NULL || temp_process->burst_time < shortest_job->burst_time) {
            if (shortest_job != NULL) {
                ret = queue_enqueue(current_core->run_queue, shortest_job);
                if (ret != 0) {
                    return ret;
                }
            }
            shortest_job = temp_process;
        }
        else {
            ret = queue_enqueue(current_core->run_queue, temp_process);
            if (ret != 0) {
                return ret;
            }

```

```

    }
}

*next_process = shortest_job;
return 0;
}

// Fixed Priority Scheduling
int schedule_prio(process_t *current_process, process_t **next_process)
{
    int len = queue_length(current_core->run_queue);
    int ret = 0;
    process_t *highest_in_queue = NULL;
    process_t *temp_process = NULL;

    // find highest priority process in the run queue
    if (len > 0) {
        for (int i = 0; i < len; i++) {
            ret = queue_dequeue(current_core->run_queue, &temp_process);
            if (ret != 0) return ret;
        }
        if (highest_in_queue == NULL || temp_process->priority > highest_in_queue->priority) {
            if (highest_in_queue != NULL) {
                ret = queue_enqueue(current_core->run_queue, highest_in_queue);
                if (ret != 0) return ret;
            }
            highest_in_queue = temp_process;
        }
        else {
            ret = queue_enqueue(current_core->run_queue, temp_process);
            if (ret != 0) return ret;
        }
    }

    // continue with current process or switch to highest priority from queue
    if (current_process && current_process->state == READY) {
        if (highest_in_queue == NULL) {
            // empty queue, continue current process
            *next_process = current_process;
            return 0;
        }
    }
}

```

```
// Compare priorities
if (current_process->priority >= highest_in_queue->priority) {
    *next_process = current_process;
    ret = queue_enqueue(current_core->run_queue,
highest_in_queue);
    return ret;
} else {
    // Switch to higher priority process
    *next_process = highest_in_queue;
    ret = queue_enqueue(current_core->run_queue,
current_process);
    return ret;
}

} else {
    // no current process, run highest priority from queue
    if (highest_in_queue == NULL) {
        // empty queue
        return -1;
    }
    // Start the highest priority process from the queue.
    *next_process = highest_in_queue;
    return 0;
}
}
```

Start von P4, P3 und P5 jeweils um 1 Zeiteinheit verschoben. -0,5
P4 startet erst bei t=6, da P1 + P2 insgesamt für 7 Einheiten rechnen.

Wie man sieht, ist es nicht möglich, das die Letzten drei Prozesse zu Problemen führen und nicht Abgeschlossen werden können. Selbst wenn man nur die Prozesse P3-5 betrachtet, überschneiden sie sich so, dass P5 nicht abgeschlossen werden kann. Wenn man Den Prozessor auf P5 warten lässt statt P4 zu starten haben wir das Problem, dass P4 nicht abgeschlossen wird.

Einfacher noch kann man sehen, dass die Gesamtrechnzeit länger als die Späteste Deadline ist.
4+3+4+2+5 = 18 > 15
Damit lassen sich die Prozesse nicht rechtzeitig ausführen.

zu a) Scheduling-Verfahren explizit nennen in Begründung. -1
z. B. Wenn Earliest-Deadline-First nicht funktioniert, dann existiert kein gültiger Ablaufplan. -1

b) Sinnvolle Zeitpunkte können sein:
1. Periodisch: Durch System-Tick sicherstellen, dass lang laufende Prozesse ihre Zeit angerechnet bekommen und ggf. verdrängt werden können.
2. Zustandswechseln: Sobald ein Prozess die CPU freiwillig abgibt oder geweckt wird, damit seine Platzierung im Red-Black-Tree korrekt ist.

Der CFS aktualisiert die vruntime über den function call update_curr(), welcher an folgenden Stellen steht:

- scheduler_tick() : Periodisch bei jedem Timer-Tick (Interval Abhängig von der HZ - Konfiguration) richtig
- dequeue_entity() / enqueue_entity() : Wenn ein Task schlafen geht oder aufwacht
- put_prev_task() : Wenn ein Task verdrängt wird oder yield() aufruft.

c)
$$vruntime = Laufzeit \times \frac{1024}{weight(nice)}$$
$$weight(-2) = 1586$$
$$vruntime = 10ms \times \frac{1024}{1586} \approx 6,46 ms$$
Also ist die vruntime von P3 nach 10ms 6,46ms richtig

Wenn bei einem Prozess die vruntime aktualisiert wird, wird sie es bei alle ihren Vorgängern auch. Das heißt das wenn bei P3 die vruntime bei 6,46ms liegt und da es der erste Prozess ist alle anderen vruntime bei 0ms sind, ist die vruntime von tg1 und "/" auch 6,46ms, also bei tg1 ist sie allerdings 10ms (standard nice wert)

Runqueue	vruntime
/	6,46 ms
tg1	6,46 ms
P1	0 ms
P2	0 ms
P3	6,46 ms

0,5/1

Prozess	CPU-Zeit
P1	500,00 ms
P2	196,17 ms
P3	303,83 ms

2/2

2,5/4 Aufgabe 3.1: Echtzeit-Scheduling und Prioritäten

0,5/2 a) Gegeben sei die folgende Menge an Rechenaufträgen mit ihren Planungsdaten:

Prozess	Ankunftszeit	Rechenzeit	Deadline
P ₁	0	4	8
P ₂	2	3	10
P ₃	6	4	14
P ₄	6	2	10
P ₅	8	5	15

Prüfen Sie, ob ein gültiger Ablaufplan für die gegebenen Aufträge existiert, wenn die Prozesse auf einem Single-Core-System nicht unterbrechend ausgeführt werden!

Hinweis: Deadline meint hier die absolute Deadline, also den entsprechenden Zeitpunkt!

Wir füllen den Plan nur Exemplarisch bei der Problematischen Stelle aus:

Prozess	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
P1	a										d					
P2			a									d				
P3						a									d	
P4							a					d				
P5								a							d	

Start von P4, P3 und P5 jeweils um 1 Zeiteinheit verschoben. -0,5
P4 startet erst bei t=6, da P1 + P2 insgesamt für 7 Einheiten rechnen.

Wie man sieht, ist es nicht möglich, das die Letzten drei Prozesse zu Problemen führen und nicht Abgeschlossen werden können. Selbst wenn man nur die Prozesse P3-5 betrachtet, überschneiden sie sich so, dass P5 nicht abgeschlossen werden kann. Wenn man Den Prozessor auf P5 warten lässt statt P4 zu starten haben wir das Problem, dass P4 nicht abgeschlossen wird.

Einfacher noch kann man sehen, dass die Gesamtrechnzeit länger als die Späteste Deadline ist.
4+3+4+2+5 = 18 > 15
Damit lassen sich die Prozesse nicht rechtzeitig ausführen.

zu a) Scheduling-Verfahren explizit nennen in Begründung. -1
z. B. Wenn Earliest-Deadline-First nicht funktioniert, dann existiert kein gültiger Ablaufplan. -1

b) Sinnvolle Zeitpunkte können sein:
1. Periodisch: Durch System-Tick sicherstellen, dass lang laufende Prozesse ihre Zeit angerechnet bekommen und ggf. verdrängt werden können.
2. Zustandswechseln: Sobald ein Prozess die CPU freiwillig abgibt oder geweckt wird, damit seine Platzierung im Red-Black-Tree korrekt ist.

Der CFS aktualisiert die vruntime über den function call update_curr(), welcher an folgenden Stellen steht:

- scheduler_tick() : Periodisch bei jedem Timer-Tick (Interval Abhängig von der HZ - Konfiguration) richtig
- dequeue_entity() / enqueue_entity() : Wenn ein Task schlafen geht oder aufwacht
- put_prev_task() : Wenn ein Task verdrängt wird oder yield() aufruft.

c)
$$vruntime = Laufzeit \times \frac{1024}{weight(nice)}$$
$$weight(-2) = 1586$$
$$vruntime = 10ms \times \frac{1024}{1586} \approx 6,46 ms$$
Also ist die vruntime von P3 nach 10ms 6,46ms richtig

Wenn bei einem Prozess die vruntime aktualisiert wird, wird sie es bei alle ihren Vorgängern auch. Das heißt das wenn bei P3 die vruntime bei 6,46ms liegt und da es der erste Prozess ist alle anderen vruntime bei 0ms sind, ist die vruntime von tg1 und "/" auch 6,46ms, also bei tg1 ist sie allerdings 10ms (standard nice wert)

Runqueue	vruntime
/	6,46 ms
tg1	6,46 ms
P1	0 ms
P2	0 ms
P3	6,46 ms

0,5/1

Prozess	CPU-Zeit
P1	500,00 ms
P2	196,17 ms
P3	303,83 ms

2/2

2,5/4 Aufgabe 3.1: Echtzeit-Scheduling und Prioritäten

0,5/2 a) Gegeben sei die folgende Menge an Rechenaufträgen mit ihren Planungsdaten:

Prozess	Ankunftszeit	Rechenzeit	Deadline
P ₁	0	4	8
P ₂	2	3	10
P ₃	6	4	14
P ₄	6	2	10
P ₅	8	5	15

Prüfen Sie, ob ein gültiger Ablaufplan für die gegebenen Aufträge existiert, wenn die Prozesse auf einem Single-Core-System nicht unterbrechend ausgeführt werden!

Hinweis: Deadline meint hier die absolute Deadline, also den entsprechenden Zeitpunkt!

Wir füllen den Plan nur Exemplarisch bei der Problematischen Stelle aus:

Prozess	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
P1	a										d					
P2			a									d				
P3						a									d	
P4							a					d				
P5								a							d	

Start von P4, P3 und P5 jeweils um 1 Zeiteinheit verschoben. -0,5
P4 startet erst bei t=6, da P1 + P2 insgesamt für 7 Einheiten rechnen.

Wie man sieht, ist es nicht möglich, das die Letzten drei Prozesse zu Problemen führen und nicht Abgeschlossen werden können. Selbst wenn man nur die Prozesse P3-5 betrachtet, überschneiden sie sich so, dass P5 nicht abgeschlossen werden kann. Wenn man Den Prozessor auf P5 warten lässt statt P4 zu starten haben wir das Problem, dass P4 nicht abgeschlossen wird.

Einfacher noch kann man sehen, dass die Gesamtrechnzeit länger als die Späteste Deadline ist.
4+3+4+2+5 = 18 > 15
Damit lassen sich die Prozesse nicht rechtzeitig ausführen.

zu a) Scheduling-Verfahren explizit nennen in Begründung. -1
z. B. Wenn Earliest-Deadline-First nicht funktioniert, dann existiert kein gültiger Ablaufplan. -1

b) Sinnvolle Zeitpunkte können sein:
1. Periodisch: Durch System-Tick sicherstellen, dass lang laufende Prozesse ihre Zeit angerechnet bekommen und ggf. verdrängt werden können.
2. Zustandswechseln: Sobald ein Prozess die CPU freiwillig abgibt oder geweckt wird, damit seine Platzierung im Red-Black-Tree korrekt ist.

Der CFS aktualisiert die vruntime über den function call update_curr(), welcher an folgenden Stellen steht:

- scheduler_tick() : Periodisch bei jedem Timer-Tick (Interval Abhängig von der HZ - Konfiguration) richtig
- dequeue_entity() / enqueue_entity() : Wenn ein Task schlafen geht oder aufwacht
- put_prev_task() : Wenn ein Task verdrängt wird oder yield() aufruft.

c)
$$vruntime = Laufzeit \times \frac{1024}{weight(nice)}$$
$$weight(-2) = 1586$$
$$vruntime = 10ms \times \frac{1024}{1586} \approx 6,46 ms$$
Also ist die vruntime von P3 nach 10ms 6,46ms richtig

Wenn bei einem Prozess die vruntime aktualisiert wird, wird sie es bei alle ihren Vorgängern auch. Das heißt das wenn bei P3 die vruntime bei 6,46ms liegt und da es der erste Prozess ist alle anderen vruntime bei 0ms sind, ist die vruntime von tg1 und "/" auch 6,46ms, also bei tg1 ist sie allerdings 10ms (standard nice wert)

Runqueue	vruntime
/	6,46 ms
tg1	6,46 ms
P1	0 ms
P2	0 ms
P3	6,46 ms

0,5/1

Prozess	CPU-Zeit
P1	500,00 ms
P2	196,17 ms
P3	303,83 ms

2/2

2,5/4 Aufgabe 3.1: Echtzeit-Scheduling und Prioritäten

0,5/2 a) Gegeben sei die folgende Menge an Rechenaufträgen mit ihren Planungsdaten:

Prozess	Ankunftszeit	Rechenzeit	Deadline
P ₁	0	4	8
P ₂	2	3	10
P ₃	6	4	14
P ₄	6	2	10
P ₅	8	5	15

Prüfen Sie, ob ein gültiger Ablaufplan für die gegebenen Aufträge existiert, wenn die Prozesse auf einem Single-Core-System nicht unterbrechend ausgeführt werden!

Hinweis: Deadline meint hier die absolute Deadline, also den entsprechenden Zeitpunkt!

Wir füllen den Plan nur Exemplarisch bei der Problematischen Stelle aus:

Prozess	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
P1	a										d					
P2			a									d				
P3						a									d	
P4							a					d				
P5								a							d	

Start von P4, P3 und P5 jeweils um 1 Zeiteinheit verschoben. -0,5
P4 startet erst bei t=6, da P1 + P2 insgesamt für 7 Einheiten rechnen.

Wie man sieht, ist es nicht möglich, das die Letzten drei Prozesse zu Problemen führen und nicht Abgeschlossen werden können. Selbst wenn man nur die Prozesse P3-5 betrachtet, überschneiden sie sich so, dass P5 nicht abgeschlossen werden kann. Wenn man Den Prozessor auf P5 warten lässt statt P4 zu starten haben wir das Problem, dass P4 nicht abgeschlossen wird.

Einfacher noch kann man sehen, dass die Gesamtrechnzeit länger als die Späteste Deadline ist.
4+3+4+2+5 = 18 > 15
Damit lassen sich die Prozesse nicht rechtzeitig ausführen.

zu a) Scheduling-Verfahren explizit nennen in Begründung. -1
z. B. Wenn Earliest-Deadline-First nicht funktioniert, dann existiert kein gültiger Ablaufplan. -1

b) Sinnvolle Zeitpunkte können sein:
1. Periodisch: Durch System-Tick sicherstellen, dass lang laufende Prozesse ihre Zeit angerechnet bekommen und ggf. verdrängt werden können.
2. Zustandswechseln: Sobald ein Prozess die CPU freiwillig abgibt oder geweckt wird, damit seine Platzierung im Red-Black-Tree korrekt ist.

Der CFS aktualisiert die vruntime über den function call update_curr(), welcher an folgenden Stellen steht:

- scheduler_tick() : Periodisch bei jedem Timer-Tick (Interval Abhängig von der HZ - Konfiguration) richtig
- dequeue_entity() / enqueue_entity() : Wenn ein Task schlafen geht oder aufwacht
- put_prev_task() : Wenn ein Task verdrängt wird oder yield() aufruft.

c)
$$vruntime = Laufzeit \times \frac{1024}{weight(nice)}$$
$$weight(-2) = 1586$$
$$vruntime = 10ms \times \frac{1024}{1586} \approx 6,46 ms$$
Also ist die vruntime von P3 nach 10ms 6,46ms richtig

Wenn bei einem Prozess die vruntime aktualisiert wird, wird sie es bei alle ihren Vorgängern auch. Das heißt das wenn bei P3 die vruntime bei 6,46ms liegt und da es der erste Prozess ist alle anderen vruntime bei 0ms sind, ist die vruntime von tg1 und "/" auch 6,46ms, also bei tg1 ist sie allerdings 10ms (standard nice wert)

Runqueue	vruntime
/	6,46 ms
tg1	6,46 ms
P1	0 ms
P2	0 ms
P3	6,46 ms

0,5/1

Prozess	CPU-Zeit
P1	500,00 ms
P2	196,17 ms
P3	303,83 ms

2/2

2,5/4 Aufgabe 3.1: Echtzeit-Scheduling und Prioritäten

0,5/2 a) Gegeben sei die folgende Menge an Rechenaufträgen mit ihren Planungsdaten:

Prozess	Ankunftszeit	Rechenzeit	Deadline
P ₁	0	4	8
P ₂	2	3	10
P ₃	6	4	14
P ₄	6	2	10
P ₅	8	5	15

Prüfen Sie, ob ein gültiger Ablaufplan für die gegebenen Aufträge existiert, wenn die Prozesse auf einem Single-Core-System nicht unterbrechend ausgeführt werden!

Hinweis: Deadline meint hier die absolute Deadline, also den entsprechenden Zeitpunkt!

Wir füllen den Plan nur Exemplarisch bei der Problematischen Stelle aus:

Prozess	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
P1	a										d					
P2			a									d				
P3						a									d	
P4							a					d				
P5								a							d	

Start von P4, P3 und P5 jeweils um 1 Zeiteinheit verschoben. -0,5
P4 startet erst bei t=6, da P1 + P2 insgesamt für 7 Einheiten rechnen.

Wie man sieht, ist es nicht möglich, das die Letzten drei Prozesse zu Problemen führen und nicht Abgeschlossen werden können. Selbst wenn man nur die Prozesse P3-5 betrachtet, überschneiden sie sich so, dass P5 nicht abgeschlossen werden kann. Wenn man Den Prozessor auf P5 warten lässt statt P4 zu starten haben wir das Problem, dass P4 nicht abgeschlossen wird.

Einfacher noch kann man sehen, dass die Gesamtrechnzeit länger als die Späteste Deadline ist.
4+3+4+2+5 = 18 > 15
Damit lassen sich die Prozesse nicht rechtzeitig ausführen.

zu a) Scheduling-Verfahren explizit nennen in Begründung. -1
z. B. Wenn Earliest-Deadline-First nicht funktioniert, dann existiert kein gültiger Ablaufplan. -1

b) Sinnvolle Zeitpunkte können sein:
1. Periodisch: Durch System-Tick sicherstellen, dass lang laufende Prozesse ihre Zeit angerechnet bekommen und ggf. verdrängt werden können.
2. Zustandswechseln: Sobald ein Prozess die CPU freiwillig abgibt oder geweckt wird, damit seine Platzierung im Red-Black-Tree korrekt ist.

Der CFS aktualisiert die vruntime über den function call update_curr(), welcher an folgenden Stellen steht:

- scheduler_tick() : Periodisch bei jedem Timer-Tick (Interval Abhängig von der HZ - Konfiguration) richtig
- dequeue_entity() / enqueue_entity() : Wenn ein Task schlafen geht oder aufwacht
- put_prev_task() : Wenn ein Task verdrängt wird oder yield() aufruft.

c)
$$vruntime = Laufzeit \times \frac{1024}{weight(nice)}$$
$$weight(-2) = 1586$$
$$vruntime = 10ms \times \frac{1024}{1586} \approx 6,46 ms$$
Also ist die vruntime von P3 nach 10ms 6,46ms richtig

Wenn bei einem Prozess die vruntime aktualisiert wird, wird sie es bei alle ihren Vorgängern auch. Das heißt das wenn bei P3 die vruntime bei 6,46ms liegt und da es der erste Prozess ist alle anderen vruntime bei 0ms sind, ist die vruntime von tg1 und "/" auch 6,46ms, also bei tg1 ist sie allerdings 10ms (standard nice wert)

Runqueue	vruntime
/	6,46 ms
tg1	6,46 ms
P1	0 ms
P2	0 ms
P3	6,46 ms

0,5/1

Prozess	CPU-Zeit
P1	500,00 ms
P2	196,17 ms
P3	303,83 ms

2/2

2,5/4 Aufgabe 3.1: Echtzeit-Scheduling und Prioritäten

0,5/2 a) Gegeben sei die folgende Menge an Rechenaufträgen mit ihren Planungsdaten:

Prozess	Ankunftszeit	Rechenzeit	Deadline
P ₁	0	4	8
P ₂	2	3	10
P ₃	6	4	14
P ₄	6	2	10
P ₅	8	5	15

Prüfen Sie, ob ein gültiger Ablaufplan für die gegebenen Aufträge existiert, wenn die Prozesse auf einem Single-Core-System nicht unterbrechend ausgeführt werden!

Hinweis: Deadline meint hier die absolute Deadline, also den entsprechenden Zeitpunkt!

Wir füllen den Plan nur Exemplarisch bei der Problematischen Stelle aus:

Prozess	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
P1	a										d					
P2			a									d				
P3						a									d	
P4							a					d				
P5								a							d	

Start von P4, P3 und P5 jeweils um 1 Zeiteinheit verschoben. -0,5
P4 startet erst bei t=6, da P1 + P2 insgesamt für 7 Einheiten rechnen.

Wie man sieht, ist es nicht möglich, das die Letzten drei Prozesse zu Problemen führen und nicht Abgeschlossen werden können. Selbst wenn man nur die Prozesse P3-5 betrachtet, überschneiden sie sich so, dass P5 nicht abgeschlossen werden kann. Wenn man Den Prozessor auf P5 warten lässt statt P4 zu starten haben wir das Problem, dass P4 nicht abgeschlossen wird.

Einfacher noch kann man sehen, dass die Gesamtrechnzeit länger als die Späteste Deadline ist.
4+3+4+2+5 = 18 > 15
Damit lassen sich die Prozesse nicht rechtzeitig ausführen.

zu a) Scheduling-Verfahren explizit nennen in Begründung. -1
z. B. Wenn Earliest-Deadline-First nicht funktioniert, dann existiert kein gültiger Ablaufplan. -1

b) Sinnvolle Zeitpunkte können sein:
1. Periodisch: Durch System-Tick sicherstellen, dass lang laufende Prozesse ihre Zeit angerechnet bekommen und ggf. verdrängt werden können.
2. Zustandswechseln: Sobald ein Prozess die CPU freiwillig abgibt oder geweckt wird, damit seine Platzierung im Red-Black-Tree korrekt ist.

Der CFS aktualisiert die vruntime über den function call update_curr(), welcher an folgenden Stellen steht:

- scheduler_tick() : Periodisch bei jedem Timer-Tick (Interval Abhängig von der HZ - Konfiguration) richtig
- dequeue_entity() / enqueue_entity() : Wenn ein Task schlafen geht oder aufwacht
- put_prev_task() : Wenn ein Task verdrängt wird oder yield() aufruft.

c)
$$vruntime = Laufzeit \times \frac{1024}{weight(nice)}$$
$$weight(-2) = 1586$$
$$vruntime = 10ms \times \frac{1024}{1586} \approx 6,46 ms$$
Also ist die vruntime von P3 nach 10ms 6,46ms richtig

Wenn bei

```
}
// by the way, saying that a yak is less likely to be an
animal than a banana, is kinda stupid but i get it

free(probability);
return 0;
}
```

b & c) 0/3: Keine Bearbeitung.

GBR Übung 04

[PDF](#)

4/4 4.1: Syscall

2/2 a)

C-Bibliothek:

```
#include <sys/time.h>

int main() {
    struct timeval tv;
    for(int i=0; i<10000000; i++) {
        gettimeofday(&tv, NULL);
    }
    return 0;
}
```

Syscall:

```
#include <sys/syscall.h>
#include <unistd.h>
#include <sys/time.h>

int main() {
    struct timeval tv;
    for(int i=0; i<10000000; i++) {
        syscall(SYS_gettimeofday, &tv, NULL);
    }
    return 0;
}
```

1/1 b)

GBR - Übung04 - Yunis Drescher (827130), Joshua Neunert (822456), Konrad Gose (828440)

- t1_main ruft t1_f2 auf. Der Stack-Frame wird an derselben Stelle wie für f1 erstellt. y=5 wird an dieselbe relative Adresse gelegt (-0x4(%rbp)).
- Gleichzeitig läuft t2_main.sleep(1), was T1 Zeit gibt, p zu setzen und k zu erhöhen (k wird 3).
- T2 dereferenziert *p. Da p auf die Adresse -0x4(%rbp) zeigt (wo früher x war und jetzt y ist), liest es den Wert 5.
- k = k * (*p) → 3 * 5 = 15.
- 0, Zufallswerte oder Segfault:
- Wenn T2 liest, nachdem t1_f2 bereits zurückgekehrt ist, zeigt p in einen "ungültigen" Stackbereich unterhalb des aktuellen Stackpointers (oder in den Frame von t1_main). Der Wert ist dann Garbage.
- Wenn T2 extrem schnell wäre (vor T1s p=&x), wäre p nicht initialisiert (Global int * p ist im BSS 0-initialisiert). Dereferenzierung von NULL führt zum Absturz (Segmentation Fault).

b)

Das Verhalten ist undefiniert, da ein "Dangling Pointer" entsteht, weil die globale Variable p mit der Adresse der lokalen Variable x belegt wird.

- Sobald t1_f1 endet (return), wird der Speicherbereich von x auf dem Stack freigegeben.
- Der Zugriff auf diesen Speicherbereich durch einen anderen Thread (t2_main) ist ein klassischer "Use-after-free"-Fehler bzw. Zugriff auf verwaisten Speicher.

4.3 1,5/4

a) 1/2

- Ausgabe 15:
 - t1_main ruft t1_f1 auf. x=3 wird auf den Stack gelegt (Offset -0x4(%rbp)). p wird auf die Adresse von x gesetzt. t1_f1 kehrt zurück.

GBR - Übung04 - Yunis Drescher (827130), Joshua Neunert (822456), Konrad Gose (828440)

```
•) time ./libc
./libc 0,14s user 0,00s system 99% cpu 0,138 total
•) time ./syscall
./syscall 0,23s user 0,58s system 99% cpu 0,819 total
```

- Das lib-Programm ist deutlich schneller als die direkte syscall-Variante.
- Bei lib fällt keine Systemzeit an.
- Beim direkten syscall dominiert die Systemzeit.

-> gettimeofday führt keinen System Aufruf aus

1/1 c)

GBR - Übung04 - Yunis Drescher (827130), Joshua Neunert (822456), Konrad Gose (828440)

Zeit	Prozess	Aktion / Codezeile	Var-Änderung	Status
ZS 1	R1	1. DOWN(mutex_str)	mutex_str = 0	OK
		2. DOWN(sem_r)	sem_r = 0	OK
		3. DOWN(mutex_rc)	mutex_rc = 0	OK
ZS 2**	R1	4. readcount++	readcount = 1	
		5. if (readcount == 1) -> DOWN(sem_w)	sem_w = 0	Writer blockiert
		6. UP(mutex_rc)	mutex_rc = 1	
Interrupt		annkunft R2 und W1		
ZS 3	R2	1. DOWN(mutex_str)		Blockiert
ZS 4	W1	1. DOWN(mutex_wc)	mutex_wc = 0	OK
		2. writecount++	writecount = 1	
		3. if (writecount == 1) -> DOWN(sem_r)		Blockiert

Wie sieht der weitere Ablauf aus, bis alle Prozesse fertig sind? -1

=> R1 hält mutex_str und sem_r. R2 wartet auf mutex_str. W1 wartet auf sem_r.

b) 0,5/2

Zeit	Prozess	Aktion / Codezeile	Var-Änderung	Status
ZS 1	W1	1. DOWN(mutex_wc)	mutex_wc = 0	OK
		2. writecount++	writecount = 1	
		3. if (writecount == 1) -> DOWN(sem_r)	sem_r = 0	Lesen gesperrt
Interrupt		annkunft R1		
ZS 2	R1	1. DOWN(mutex_str)	mutex_str = 0	OK
		2. DOWN(sem_r)		Blockiert

GBR - Übung04 - Yunis Drescher (827130), Joshua Neunert (822456), Konrad Gose (828440)

```
lib
Breakpoint 1, 0x00007ffff7fc48b0 in gettimeofday ()
(gdb) step
Single stepping until exit from function gettimeofday,
which has no line number information.
0x000055555555009a in main ()
(gdb) dis
disable      disassemble  disconnect  display
(gdb) dis
disable      disassemble  disconnect  display
(gdb) disassemble
Dump of assembler code for function main:
0x00005555555500a0 <+0>:   push    %rsp
0x00005555555500a1 <+1>:   mov     %rsp,%rbp
0x00005555555500a4 <+4>:   push    %r12
0x00005555555500a6 <+6>:   push    %rbx
0x00005555555500a7 <+7>:   lea     -0x30(%rbp),%r12
0x00005555555500ab <+11>:  mov     $0x20,%r10
0x00005555555500af <+15>:  mov     %r10,%rbx
0x00005555555500b8 <+24>:  mov     %rbx,-0x18(%rbp)
0x00005555555500b9 <+28>:  mov     $0x9b9680,%ebx
0x00005555555500c1 <+33>:  data16 cs nopw 0x0(%rax,%rax,1)
0x00005555555500c8 <+44>:  nopl    0x0(%rax)
0x0000555555550090 <+48>:  xor     %eax,%eax
0x0000555555550092 <+50>:  mov     %r12,%rdi
0x0000555555550095 <+53>:  call    0x555555550040 <gettimeofday@plt>
=> 0x000055555555009a <+58>:  sub     $0x1,%ebx
0x000055555555009d <+61>:  jne     0x555555550099 <main+48>
0x000055555555009f <+63>:  mov     -0x18(%rbp),%rax
0x00005555555500a3 <+67>:  sub     %fs:0x28,%rax
0x00005555555500ac <+76>:  jne     0x5555555500b9 <main+89>
0x00005555555500ae <+78>:  add     $0x20,%rsp
0x00005555555500b2 <+82>:  xor     %eax,%eax
0x00005555555500b4 <+84>:  pop     %rbx
0x00005555555500b5 <+85>:  pop     %r12
0x00005555555500b7 <+87>:  pop     %rbp
0x00005555555500b8 <+88>:  ret
0x00005555555500b9 <+89>:  call    0x555555550030 <__stack_chk_fail@plt>
End of assembler dump.
```

GBR - Übung04 - Yunis Drescher (827130), Joshua Neunert (822456), Konrad Gose (828440)

W2 trifft während der zweiten Zeitscheibe ein und wird dadurch hinter W1 eingeordnet, sodass W1 vorher rechnen darf. -0.5

Zeit	Prozess	Aktion / Codezeile	Var-Änderung	Status
Interrupt		annkunft W2		
ZS 3	W2	1. DOWN(mutex_wc)		Blockiert

Analogue wie bei a): Angabe der Schritte bis alle Prozesse fertig sind. -1

3/3 4.5

2/2 a)

```
ls -la
dr-xr-xr-x  2 joshua users  4 B Sat Dec 13 17:51:45 2025 .
dr-xr-xr-x  2 joshua users  0 B Sat Dec 13 17:51:38 2025 ..
lrwxrwxrwx  1 joshua users 64 B Sat Dec 13 17:51:47 2025 20 -> /dev/null
lrwxrwxrwx  1 joshua users 64 B Sat Dec 13 17:51:47 2025 21 -> /dev/pts/0
lrwxrwxrwx  1 joshua users 64 B Sat Dec 13 17:51:47 2025 22 -> /tmp/err_uid
lrwxrwxrwx  1 joshua users 64 B Sat Dec 13 17:51:47 2025 23 -> /tmp/tempfileT0uMNe

ls -la
dr-xr-xr-x  2 joshua users  3 B Sat Dec 13 17:51:45 2025 .
dr-xr-xr-x  2 joshua users  0 B Sat Dec 13 17:51:38 2025 ..
lrwxrwxrwx  1 joshua users 64 B Sat Dec 13 17:51:47 2025 20 -> /dev/null
lrwxrwxrwx  1 joshua users 64 B Sat Dec 13 17:51:47 2025 21 -> /dev/pts/0
lrwxrwxrwx  1 joshua users 64 B Sat Dec 13 17:51:47 2025 22 -> /tmp/err_uid
```

Unter /proc/PID/fd/ befindet:

- 0: Verweist auf /dev/null – Standard Input.
- 1: Verweist auf Terminal (/dev/pts/X) – Standard Output.
- 2: Verweist auf /tmp/err_uid – Standard Error.
- 3: Verweist auf die temp Datei /tmp/tempfileXXXXXX.
 - mkstemp(str3) erstellt einen Dateideskriptor. Da 0, 1 und 2 belegt sind, ist 3 der nächste freie.

1/1 b)

GBR - Übung04 - Yunis Drescher (827130), Joshua Neunert (822456), Konrad Gose (828440)


```
[nix-shell:/proc/$2215/task]$ tree */fd
52215/fd
├── 0 → /dev/null
├── 1 → /dev/pts/0
├── 2 → /tmp/err_uid
└── 3 → /tmp/tempfile648UVX

52216/fd
├── 0 → /dev/null
├── 1 → /dev/pts/0
├── 2 → /tmp/err_uid
└── 3 → /tmp/tempfile648UVX

2 directories, 8 files

[nix-shell:/proc/$2215/task]$ tree */fd
52215/fd
├── 0 → /dev/null
├── 1 → /dev/pts/0
├── 2 → /tmp/err_uid
└── 3 → /tmp/tempfile648UVX

52216/fd
├── 0 → /dev/null
├── 1 → /dev/pts/0
└── 2 → /tmp/err_uid

2 directories, 6 files
```

- Unterscheiden sich diese während der Ausführung des Programms?
 - Nein. Threads, die mit `pthread_create` erstellt werden, teilen sich (via `CLONE_FILES` Flag beim `clone`-Syscall) dieselbe Dateideskriptortabelle.
 - Wenn man `/proc/PID/task/TID/fd` für den Hauptthread und den Worker-Thread ansieht, sind die Einträge identisch.
- Fork statt Thread:
 - Wenn statt `pthread_create` ein `fork()` verwendet wird, würde der Kindprozess eine Kopie der Dateideskriptortabelle erhalten.

- Wenn der Elternprozess dann `close(fptr)` aufruft, wird nur sein Eintrag entfernt.
- Der Kindprozess hätte den Dateideskriptor weiterhin offen und könnte erfolgreich schreiben.
- => Prozesse haben getrennte Adressräume und Ressourcentabellen (Copy-on-Write bzw. Duplizierung beim Start), Threads teilen sich diese Ressourcen.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <sys/mman.h>
5 #include <unistd.h>
6 #include <semaphore.h>
7 #include <time.h>
8 #include <errno.h>
9
10 #define SHM_NAME "/arche_noah_shm"
11 #define SEM_MUTEX "/arche_noah_mutex"
12 #define SEM_FULL "/arche_noah_full"
13 #define N 10
14
15 struct shared_data
16 {
17     int bookings[N];
18     int count;
19 };
20
21 void sem_wait_intr(sem_t *sem)
22 {
23     while (sem_wait(sem) == -1)
24     {
25         if (errno != EINTR)
26         {
27             perror("sem_wait");
28             exit(1);
29         }
30     }
31 }
32
33 int already_used(struct shared_data *data, int id)
34 {
35     for (int i = 0; i < N; i++)
36     {
37         if (data->bookings[i] == id)
38             return 1;
39     }
40     return 0;
41 }
42
43 int main()
44 {
45     srand(time(NULL) * getpid());
46
47     int shm_fd = shm_open(SHM_NAME, O_RDWR | O_CREAT, 0666);
48     if (shm_fd == -1)
49         fprintf(stderr, "shm_open failed: %s\n", strerror(errno));
50
51     struct shared_data *data = mmap(NULL, sizeof(*data),
52                                     PROT_READ | PROT_WRITE,
53                                     MAP_SHARED, shm_fd, 0);
54
55     sem_t *mutex = sem_open(SEM_MUTEX, O_CREAT, 0666, 1);
56     sem_t *full = sem_open(SEM_FULL, O_CREAT, 0666, 0);
57
58     if (mutex == SEM_FAILED || full == SEM_FAILED)
59     {
60         perror("sem_open");
61         exit(1);
62     }
63
64     sem_wait_intr(mutex);
65
66     if (data->count == N)
67     {
68         sem_post(mutex);
69     }
70 }
```

```
68     return 0;
69 }
70
71 int id;
72 do
73 {
74     id = rand() % 10000 + 1;
75 } while (already_used(data, id));
76
77 data->bookings[data->count] = id;
78 data->count++;
79
80 printf("Client %d reserves Space %d\n", id, data->count);
81
82 if (data->count == N)
83 {
84     sem_post(full);
85 }
86
87 sem_post(mutex);
88
89 mmap(data, sizeof(*data));
90 close(shm_fd);
91
92 return 0;
93 }
94
```

GBR - Übung04 - Yunis Drescher (827130), Joshua Neunert (822456), Konrad Gose (828440)

GBR - Übung04 - Yunis Drescher (827130), Joshua Neunert (822456), Konrad Gose (828440)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <sys/mman.h>
5 #include <unistd.h>
6 #include <semaphore.h>
7 #include <errno.h>
8
9 #define SHM_NAME "/arche_noah_shm"
10 #define SEM_MUTEX "/arche_noah_mutex"
11 #define SEM_FULL "/arche_noah_full"
12 #define N 10
13
14 struct shared_data
15 {
16     int bookings[N];
17     int count;
18 };
19
20 void sem_wait_intr(sem_t *sem)
21 {
22     while (sem_wait(sem) == -1)
23     {
24         if (errno != EINTR)
25         {
26             perror("sem_wait");
27             exit(1);
28         }
29     }
30 }
31
32 int main()
33 {
34     int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
35     if (shm_fd == -1)
36     {
37         perror("shm_open");
38         exit(1);
39     }
40
41     ftruncate(shm_fd, sizeof(struct shared_data));
42
43     struct shared_data *data = mmap(NULL, sizeof(*data),
44                                     PROT_READ | PROT_WRITE,
45                                     MAP_SHARED, shm_fd, 0);
46
47     sem_t *mutex = sem_open(SEM_MUTEX, O_CREAT, 0666, 1);
48     sem_t *full = sem_open(SEM_FULL, O_CREAT, 0666, 0);
49
50     if (mutex == SEM_FAILED || full == SEM_FAILED)
51     {
52         perror("sem_open");
53         exit(1);
54     }
55
56     printf("Master waits for complete Buchungsliste...\n");
57
58     sem_wait_intr(full);
59
60     sem_wait_intr(mutex);
61     printf("Buchungsliste complete:\n");
62     for (int i = 0; i < N; i++)
63     {
64         printf("Space %d: Client %d\n", i, data->bookings[i]);
65     }
66     sem_post(mutex);
67 }
```

```
68     mmap(data, sizeof(*data));
69     close(shm_fd);
70
71     shm_unlink(SHM_NAME);
72     sem_unlink(SEM_MUTEX);
73     sem_unlink(SEM_FULL);
74
75     return 0;
76 }
77
```

list.c

```
/*
 * Simple double linked list implementation
 * created only for educational purposes
 *
 * Task: Operations should be extended with pthread_rwlocklocks
 */
#include "list.h"
#include <stdlib.h>

/*
 * function: dl_list_create
 * parameter: dl_list ** list: Pointer to the Pointer that should hold the address of the list
 * returns: integer, 0 success, -1 malloc failure
 * description: allocates and initializes the double linked list structure
 */
int dl_list_create(dl_list ** list)
{
    *list = calloc(1, sizeof(dl_list));
    if (*list == NULL) {
        return -1;
    }

    // Initialize RW lock
    if (pthread_rwlock_init(&(*list)->lock, NULL) != 0) {
        free(*list);
        return -1;
    }
    return 0;
}

/*
 * function: dl_list_free
 * parameter: dl_list * list: pointer to the list
 * returns: void
 * description: cleans up all structures
 */
void dl_list_free(dl_list *list)
{
    // There is no spoon
    if (list == NULL) {
        return;
    }
    while (!dl_list_remove(list, 0)) {}
}
```

```
// Destroy RW lock
pthread_rwlock_destroy(&list->lock);
free(list);

/*
 * function: dl_list_insert
 * parameter: dl_list * list: pointer to the list, int value: value to be added, int position
 * returns: int: 0 on success, -1 on position out of range, -2 malloc failure, -3 on other error
 * description: inserts an value at position in the list
 */
int dl_list_insert(dl_list *list, int value, int position)
{
    if (list == NULL || position < 0) {
        return -3;
    }

    // lock for writing
    pthread_rwlock_lock(&list->lock);

    dl_list_element *list_iterator = list->first; // list element pointer
    //iterate to the right position
    while (position > 0; list_iterator = list_iterator->next; position--) {}

    if (position > 1 || (position == 1 && list_iterator == NULL)) {
        pthread_rwlock_unlock(&list->lock);
        return -1;
    }

    //allocate new element
    dl_list_element *new_element = calloc(1, sizeof(dl_list_element));
    if (new_element == NULL) {
        pthread_rwlock_unlock(&list->lock);
        return -2;
    }
    new_element->value = value;

    //use as first element
    if (position == 0) {
        new_element->next = list->first;
        list->first = new_element;
    } else {
}
```

```
new_element->next=list_iterator->next;
new_element->prev=list_iterator;
list_iterator->next=new_element;
}

// do we have a next element
if(new_element->next==NULL) {
    new_element->next=&prev_new_element;
} else {
    // are we last element
    list->last=new_element;
}

// unlock and return success
pthread_rwlock_unlock(&list->lock);
return 0;
}

/*
 * function: dllist_get
 * parameter: dllist * list: pointer to the list, int position: list position the value is
 * returns: int: 0 on success, -1 on position out of range, -2 on other errors
 * description: gets an element of the list at the position
 */
int dllist_get(dllist* list, int position, int* result)
{
    if ( list == NULL || position <= 0 ) {
        return -2;
    }

    // lock for reading
    pthread_rwlock_rdlock(&list->lock);

    dllist_element* list_iterator= list->first; // list element pointer

    while(position > 0 && list_iterator !=NULL) {
        list_iterator=list_iterator->next;
        position--;
    }

    if (position==0 || list_iterator == NULL) {
        // unlock and return error
        pthread_rwlock_unlock(&list->lock);
        return -1;
    }
}
```

```
*result=list_iterator->value;

// unlock and return success
pthread_rwlock_unlock(&list->lock);
return 0;
}

/*
 * function: dllist_remove
 * parameter: dllist * list: pointer to the list, int position: list position the value is
 * returns: int: 0 on success, -1 on position out of range, -2 on other errors
 * description: removes an list element at the position
 */
int dllist_remove(dllist* list, int position)
{
    if ( list == NULL || position <= 0 ) {
        return -3;
    }

    // lock for writing
    pthread_rwlock_wrlock(&list->lock);

    dllist_element* list_iterator= list->first; // list element pointer
    //iterate to the right position
    while(position > 0 && list_iterator !=NULL) {
        list_iterator=list_iterator->next;
        position--;
    }

    if (position!=0 || list_iterator == NULL) {
        // unlock and return error
        pthread_rwlock_unlock(&list->lock);
        return -1;
    }

    //is there an element before us
    if (list_iterator->prev != NULL) {
        list_iterator->prev->next= list_iterator->next;
    } else {
        list->first=list_iterator->next;
    }

    if(list_iterator->next != NULL) {
        list_iterator->next->prev= list_iterator->prev;
    } else {
        list->last=list_iterator->prev;
    }
}
```

```
}

free(list_iterator);

// unlock and return success
pthread_rwlock_unlock(&list->lock);

return 0;
}

test.c
/*
 * Simple test application for the linked list application.
 * created only for educational purposes
 *
 * Task: This test application need to modified that the linked list is tested in multiple
 */

#include "list.h"
#include <stdio.h>

void print_list(dllist *list){
    int v,r;
    int i=0;
    while((r=dllist_get(list,i,&v))) {
        printf("%d-",v);
        i++;
    }
    printf("\n");
}

int main()
{
    int toadd[10]= {7,0,3,0,2,1,9,2,8,4};
    int torem[4]= {0,4,1,0};
    int r;

    dllist* list;
    if(dllist_create(&list)) {
        fprintf(stderr,"Couldn't create list");
    }
}
```

```
for(int i = 0; i < 10; i+=2) {
    if(dllist_insert(list,toadd[i],toadd[i+1])) {
        printf("couldn't add %d at %d\n",toadd[i],toadd[i+1]);
    }
}

print_list(list);

for(int k = 0; k < 4; k+=1) {
    printf("trying to remove element at position: %d \n", torem[k]);
    if(r=dllist_remove(list,torem[k])) {
        printf("couldn't remove element at %d ret: %d \n",torem[k],r);
    }
    print_list(list);
}

for(int i = 0; i < 3; i+=2) {
    if(dllist_insert(list,toadd[i],toadd[i+1])) {
        printf("couldn't add %d at %d\n",toadd[i],toadd[i+1]);
    }
}

print_list(list);

dllist_free(list);
}
```

test_all.c nicht vorhanden 0/2

Übungsblatt 5

Montag, 5. Januar 2020 20:38

Yunis Drescher, 827130 Joshua Neunert, 822456 Konrad Gose, 828440

Aufgabe 5.2: Synchronisation

- a) Worin besteht der Unterschied zwischen dem Java- Monitor- und dem Hoare-Monitorkonzept?
- b) Die folgende Umsetzung der put- und get-Funktionen des Java-Cubbyhole-Beispiels ist fehlerhaft. Wie kann man den Fehler beheben?

```
public synchronized int get() {
    if (available == false) {
        try {
            wait();
        } catch (InterruptedException e) {}
    }
    available = false;
    notify();
    return contents;
}

public synchronized void put(int value) {
    if (available == true) {
        try {
            wait();
        } catch (InterruptedException e) {}
    }
    contents = value;
    available = true;
    notify();
}
```

- c) Überdies kann es auch in der korrigierten Lösung zu Livelocks kommen. Geben Sie ein Beispielszenario dafür an!

(1 + 1 + 1 Punkte)

a) Beim Java-Monitor wird durch notify() ein Thread wieder bereit gemacht, aber nicht direkt ausgeführt. Der Thread der notify() ausgeführt hat neht den Monitor solange bis er ihn verlässt. Der geweckte Thread arbeitet also irgendwann später.

Beim Hoare-Monitor wird der Thread der durch signal(k) aufgerufen wird sofort aktiv und der Thread der signal(s) aufruft wird blockiert. Wenn der geweckte Thread den Monitor verlässt, kann der Signal sendende Thread weiter arbeiten. Dadurch entsteht der Unterschied, dass beim Hoare-Monitor der geweckte Thread garantiert seine Bedingung erfüllt hat, während beim Java-Monitor die Bedingung sich wieder geändert haben kann. Daher muss beim Java-Monitor immer wieder mit while die Bedingung geprüft werden.

Hoare verschiedene Bedingungsvariablen mit eigener WS.

Java nur eine WS

0.5/1

c) Da es sich um Java handelt, müssen wir die Verfügbarkeit nicht nur einmal mit "if" prüfen sondern mit einer while-schleife. Also:

Zeile 2: while (available == false) {}
Zeile 12: while (available == true) {}

Der Rest des codes kann gleich bleiben.

1/1

d) Gehen wir davon aus das es mehr als zwei Threads gibt. Da nur notify() und nicht notifyAll() genutzt wird, kann es sein dass ein Thread geweckt wird der direkt wieder blockiert wird. Dieser Thread kann dann kein neues notify() ausführen und alles steht still. Beispiel:
P1, P2, P3
P1, P2 wollen put() nutzen, P2 will get() nutzen.
Wenn available = false, dann wartet P3 in get(). P1 ruft nun put() auf und schreibt seinen wert. Danach setzt er available = true und notify() weckt P2. P2 sieht available = true und geht in wait. Nun ist P1 abgeschlossen und P2 und P3 stecken in der wait() schleife.

1/1

Aufgabe 5.3: Buddy-System

Ein Betriebssystem verwende das Buddy-System für die Speicherverwaltung mit 2^k KB (k ≥ 0) großen Blöcken. Der Rechner ist mit 2048 KByte RAM ausgestattet, die vollständig in Buddy-System verwaltet werden.

Die Prozesse A, B, C und D fordern Speicher in folgender Reihenfolge an:

- i) A fordert 236 KByte an.
ii) B fordert 144 KByte an.
iii) C fordert 76 KByte an.
iv) D fordert 456 KByte an.

- a) Zeichnen Sie, nach den Schritten ii) , iii) und iv) die Speicherbelegung ausgehend vom komplett freien und nicht in Blöcke eingeteilten Speicher.
- b) Wieviele Blöcke sind noch übrig? Wie groß ist dann der größte verfügbare Speicherbereich?

- c) Geben Sie den aktuellen Verschnitt an!

Nacheinander geben dann die Prozesse B, A, D und C in dieser Reihenfolge ihren Speicher frei.

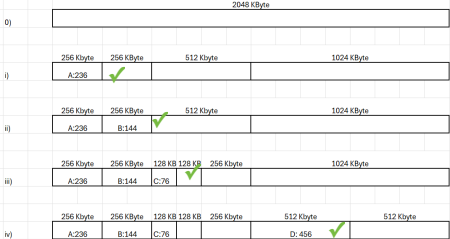
- d) Zeichnen Sie, ausgehend vom Ergebnis von a), für jeden Schritt die Speicherbelegung. Geben Sie nach jeder Speicherfreigabe den größten zur Verfügung stehenden Block an!

a)

Aufgabe 5.3: 4/4

Aktion:

a) 2/2



0,5/0,5

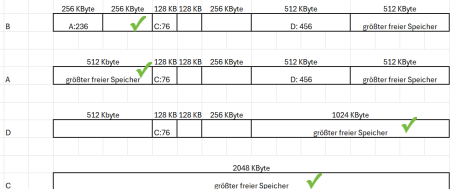
- b) Es sind noch 3 Blöcke übrig, wovon der größte Block 512 KByte groß ist.

0,5/0,5

- c) Der Verschnitt ist der ungenutzte Speicher in jedem Benutzten Speicherblock, also:
(256 - 236) + (256 - 144) + (128 - 76) + (512 - 456) = 240

1/1

d) Entfernen:



Aufgabe 5.4: Gutartigkeit

Zeigen Sie, daß LRU ein gutartiger Seitenersatzalgorithmus ist!

(2Punkte)

Beweis über Stack-Algorithmus:

Ein Algorithmus ist gutartig, wenn er ein Stack-Algorithmus ist. Ein Algorithmus ist ein Stack-Algorithmus, wenn es eine feste Prioritätsordnung gibt mit der die Seiten eingeladen werden.

Also gilt in Bezug auf Gutartigkeit, dass:
Der Speicher der Größe n enthält immer die m höchstpriorisierten Seiten.
Bei größerem Speicher werden nur "unten" zusätzliche Seiten ergänzt.

Da LRU eine feste-Prioritätsordnung hat, höchste Priorität der Ersetzung für die am längsten nicht genutzte Seite und somit die Anforderung eines Stack-Algorithmus erfüllt ist er gutartig

Aufgabe 5.5: Speicherverwaltung Seitenfehler 2/2

Die Behandlung eines Seitenfehlers erfordert i. A. Maßnahmen zur Ersetzung einer Seite in einem Seitenrahmen (Kachel) des Arbeitsspeichers. Dazu muss zunächst eine andere Seite aus dem Arbeitsspeicher verdrängt werden.

Bei der Ausführung eines Benutzerprogramms tritt folgende Sequenz von Seitenzugriffen auf:

7 6 2 7 5 5 3 2 7 4 5

Bestimmen Sie die minimale Anzahl der Seitenfehler, die auftreten kann! Gehen Sie dabei von 4 Seitenrahmen aus und begründen Sie Ihre Antwort. (2 Punkte)

Die Minimale Anzahl von Seitenfehler erhält man über den Belady-Algorithmus, da er die Seiten die man als nächstes braucht kennt und somit immer optimal löscht. Dieser ist aber nur möglich wenn man die Sequenz kennt bevor sie passiert.

SW	7	6	2	7	5	5	3	2	7	4	5
1	7	7	7	7	7	7	7	7	7	4	4
2		6	6	6	6	6	3	3	3	3	3
3			2	2	2	2	2	2	2	2	2
4					5	5	5	5	5	5	5
SF	+	+	+	+	+	+	+	+	+	+	+

Die minimale Anzahl der Seitenfehler ist also 6

Source (IP:Port)	Destination (IP:Port)	Packets	Bytes
10.3.10.45:43044	10.3.10.43:80	2,431	3,561,309
10.3.10.45:43058	10.3.10.43:80	10	1,638



UDP

Source (IP:Port)	Destination (IP:Port)	Packets	Bytes
10.3.10.45:38025	10.3.10.43:53	4	336
10.3.10.45:58386	10.3.10.43:53	4	336



c) 0/1

0,348790

+ 0,000012

+ 0,815977

+ 0,000024

+ 0,000139

+ 0,000054

+ 0,000113

+ 0,000041

1,16515 / 8 ~= 0,14564375s

Durchschnittliche Latenzzeit für
DNS-Anfragen: 0,16 ms -0,5

In Wireshark: Statistics -> DNS
-> Query Responses

http: 0.000029s

Latenz des HTTP-Requests: 0,179588 s
Pakete 14 und 2436 betrachten. -0,5

d) 1/1